Patterns

CellPress
OPEN ACCESS

Tutorial

Avoiding common machine learning pitfalls

Michael A. Lones[1,*]
[1]School of Mathematical and Computer Sciences, Heriot-Watt University, Edinburgh, UK
*Correspondence: m.lones@hw.ac.uk
https://doi.org/10.1016/j.patter.2024.101046

THE BIGGER PICTURE Machine learning has transitioned from a niche pursuit to one with mass appeal. Thanks to the accessibility of modern machine learning tools, it is now very easy to get started in machine learning, yet this ease of use masks the underlying complexities of doing machine learning. This, coupled with a relatively inexperienced community of practitioners, has led to flawed practices, which are reflected in issues such as poor reproducibility within machine-learning-based studies.

This tutorial aims to address this problem by educating practitioners about the many things that can go wrong when applying machine learning and providing guidance on how to avoid these pitfalls. However, this is just part of the longer-term process that is needed to improve practice, as machine learning will only meet its ambitions if it is able to become a robust and trusted applied discipline. Other factors that have a role to play in this include better tools, standardization, and regulation.

SUMMARY

Mistakes in machine learning practice are commonplace and can result in loss of confidence in the findings and products of machine learning. This tutorial outlines common mistakes that occur when using machine learning and what can be done to avoid them. While it should be accessible to anyone with a basic understanding of machine learning techniques, it focuses on issues that are of particular concern within academic research, such as the need to make rigorous comparisons and reach valid conclusions. It covers five stages of the machine learning process: what to do before model building, how to reliably build models, how to robustly evaluate models, how to compare models fairly, and how to report results.

INTRODUCTION

It is easy to make mistakes when applying machine learning (ML), and these mistakes can result in ML models that fail to work as expected when applied to data not seen during training and testing.[1] This is a problem for practitioners, as it leads to the failure of ML projects. However, it is also a problem for society, as it erodes trust in the findings and products of ML.[2]

This guide aims to help newcomers avoid some of these mistakes. It is written by an academic and focuses on lessons learnt while doing ML research in academia. While primarily aimed at students and scientific researchers, it should be accessible to anyone getting started in ML and only assumes a basic knowledge of ML techniques. However, unlike similar guides aimed at a more general audience, it includes topics that are of a particular concern to academia, such as the need to rigorously evaluate and compare models in order to get work published.

To make it more readable, the guidance is written informally, in a "dos and do nots" style. It is not intended to be exhaustive, and references are provided for further reading; for publications with access restrictions, citations to preprints are also included where available. Since it does not cover issues specific to particular academic subjects, it is recommended that readers also consult subject-specific guidance where available, e.g., in clinical medicine,[3] genomics,[4] environmental research,[5] materials science,[6] business and marketing,[7] computer security,[8] and social science.[9]

The tutorial is divided into five sections. "Before you start to build models" covers issues that can occur early in the ML process and focuses on the correct use of data and adequate consideration of the context in which ML is being applied. "How to reliably build models" then covers pitfalls that occur during the selection and training of models and their components. "How to robustly evaluate models" presents pitfalls that can lead to an incorrect understanding of model performance. "How to compare models fairly" then extends this to the situation where models are being compared, discussing how common pitfalls can lead to misleading findings. "How to report your results" focuses on reproducibility and factors that can lead to incomplete or deceptive reporting.

ML pitfalls are not static and continue to evolve as ML develops. To address this, the preprint version of this tutorial[10] has been updated annually since it was first released in 2021, and it will continue to be updated in the future.

This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

BEFORE YOU START TO BUILD MODELS

It is normal to want to rush into training and evaluating models, but it is important to take the time to think about the goals of a project, fully understand the data that will be used to support these goals, consider any limitations of the data that need to be addressed, and understand what has already been done in your field. If you do not do these things, then you may end up with results that are hard to publish or models that are not appropriate for their intended purpose.

Do think about how and where you will use data

Data are central to most ML projects but are often in short supply. Therefore, it is important to think carefully about what data you need and how and where you will use them. Abstractly, you need data for two things: training models and testing models. However, for various reasons, this does not necessarily translate into using a single dataset divided into two parts. To begin with, model development often involves a period of experimentation: trying out different models with different hyperparameters and preprocessing the data in different ways. To avoid overfitting (see "do avoid sequential overfitting"), this process requires a separate validation set, i.e., an additional set of training data that are not used directly in training or testing models. If you have no prior idea of what modeling approach you are going to use, then this experimentation phase could potentially involve a lot of comparisons. Due to the multiplicity effect (see "do correct for multiple comparisons"), the more comparisons you do, the more likely you are to overfit the validation data, and so the less useful the validation set will become in guiding your modeling decisions. So, in practice, you might want to set aside multiple validation sets for this. Then, there is the question of how you adequately test your selected model. Because it has the same biases as the training data, a test set taken from the same dataset as the training data may not be sufficient to measure the model's generality—see "do use an appropriate test set" and "do report performance in multiple ways" for more on this—meaning that, in practice, you may need more than one test dataset to robustly evaluate your model. Also, be aware that you will often need additional test data when using cross-validation (CV); see "do save some data to evaluate your final model instance."

Do take the time to understand your data

Eventually, you will want to publish your work. This is a lot easier to do if your data are from a reliable source, have been collected using a reliable methodology, and are of good quality. For instance, if you are using data collected from an internet resource, make sure you know where they came from. Are they described in a paper? If so, take a look at the paper; make sure it was published somewhere reputable, and check whether the authors mention any limitations of the data. Do not assume that because a dataset has been used by a number of papers, it is of good quality—sometimes data are used just because they are easy to get hold of, and some widely used datasets are known to have significant limitations (see Paullada et al.[11] for a discussion of this). If you train your model using bad data, then you will most likely generate a bad model: a process known as "garbage in garbage out." One way to avoid bad datasets is to build a direct relationship with people who generate data, as this increases the likelihood of obtaining a good-quality dataset that meets your needs. It also avoids problems of overfitting community benchmarks; see "do not always believe results from community benchmarks." Yet, regardless of where your data comes from, always begin by making sure that your data make sense. Do some exploratory data analysis (see Cox[12] for suggestions). Look for missing or inconsistent records. It is much easier to do this now, before you train a model, rather than later, when you are trying to explain to reviewers why you used bad data.

Do not look at all of your data

As you look at data, it is quite likely that you will spot patterns and make insights that guide your modeling. This is another good reason to look at data. However, it is important that you do not make untestable assumptions that will later feed into your model. The "untestable" bit is important here; it is fine to make assumptions, but these should only feed into the training of the model, not the testing. So, to ensure that this is the case, you should avoid looking closely at any test data in the initial exploratory analysis stage. Otherwise, you might, consciously or unconsciously, make assumptions that limit the generality of your model in an untestable way. This is a theme I will return to several times since the leakage of information from the test set into the training process is a common reason why ML models fail to generalize. See "do not allow test data to leak into the training process" for more on this.

Do clean your data

Even good-quality datasets will have issues. Some of these come from unavoidable noise or omissions in the data collection process, while others are due to human error during collection or collation. Whatever the cause, it is important to identify any issues, and do this before you start to build models. One common problem to look out for is data duplication, i.e., the unintentional inclusion of multiple copies of a data point. This can cause serious problems when a model is evaluated (see "do not do data augmentation before splitting your data" for an example) and so should be identified and removed early on. Another common problem is missing values. Some models can cope with these, but many cannot, and so you will have to replace missing values with something else before they can be trained. There are various forms of imputation that can be used to achieve this; see Emmanuel et al.[13] for a review. If you do imputation, be careful to avoid data leaks during imputation—see "do not allow test data to leak into the training process." You should also check for outliers in your data, but only remove these if they are likely to be the result of noise or error rather than being natural extremes of the underlying data-generating process. For example, if a person's age is greater than 150, then it is probably an error; if it is 110, then it could be a natural outlier. A related issue is meaningless or inconsistent data, for instance a person with a negative age. Data cleaning can be a time-consuming process and becomes more challenging as the complexity of data increases. For this reason, many people have explored automating data cleaning using ML approaches; see Côté et al.[14,15] for a review.

Do make sure you have enough data

If you do not have enough data, then it may not be possible to train a model that generalizes. Working out whether this is the case can be challenging and may not be evident until you start building models: it all depends on the signal-to-noise ratio in the dataset. If the signal is strong, then you can get away with less data; if it is weak, then you need more data. If you cannot get more data—and this is a common issue in many research fields—then you can try using data augmentation techniques (see Wang et al.[16] and, for time-series data, Iglesias et al.[17]). These can be quite effective for boosting small datasets, though do not do data augmentation before splitting your data. Data augmentation is also useful in situations where you have limited data in certain parts of your dataset, e.g., in classification problems where you have less samples in some classes than others, a situation known as class imbalance. See Haixiang et al.[18] for a review of methods for dealing with this; also see "do choose metrics carefully." Another option for dealing with small datasets is to use transfer learning—see "do keep up with progress in deep learning (and its pitfalls)." A danger when using small datasets is that different data partitions may be biased, for instance in terms of the quality or difficulty of data they contain. For this reason, it is advisable to consider frequent repartitioning. CV (see "do evaluate a model multiple times") is an efficient way of achieving this in small datasets. If you have limited data, then it is also likely that you will have to limit the complexity of the ML models you use, as models with many parameters, like deep neural networks, can easily overfit small datasets (see "do not assume deep learning will be the best approach"). Regardless of how you handle the problem of limited data, it is important to identify this issue early on and come up with a suitable strategy to mitigate against it.

Do talk to domain experts

Domain experts can be very valuable. They can help you to understand which problems are useful to solve, choose the most appropriate feature set and ML model to use, and publish to the most appropriate audience. Failing to consider the opinion of domain experts can lead to projects that do not solve useful problems or that solve useful problems in inappropriate ways. An example of the latter is using an opaque ML model to solve a problem where there is a strong need to understand how the model reaches an outcome, e.g., in making medical or financial decisions.[19] At the beginning of a project, domain experts can help you to understand the data and point you toward features that are likely to be predictive. At the end of a project, they can help you to publish in domain-specific journals and, hence, reach an audience that is most likely to benefit from your research.

Do survey the literature

You are probably not the first person to throw ML at a particular problem domain, so it is important to understand what has and has not been done previously. Other people having worked on the same problem is not a bad thing; academic progress is typically an iterative process, with each study providing information that can guide the next. It may be discouraging to find that someone has already explored your great idea, but they most likely left plenty of avenues of investigation still open, and their previous work can be used as justification for your work. To ignore previous studies is to potentially miss out on valuable information. For example, someone may have tried your proposed approach before and found fundamental reasons why it will not work (and therefore saved you a few years of frustration), or they may have partially solved the problem in a way that you can build on. So, it is important to do a literature review before you start work; leaving it too late may mean that you are left scrambling to explain why you are covering the same ground or not building on existing knowledge when you come to write a paper.

Do think about how your model will be deployed

Why do you want to build an ML model? This is an important question, and the answer should influence the process you use to develop your model. Many academic studies are just that—studies—and not really intended to produce models that will be used in the real world. This is fair enough, as the process of building and analyzing models can itself give very useful insights into a problem. However, for many academic studies, the eventual goal is to produce an ML model that can be deployed in a real-world situation. If this is the case, then it is worth thinking early on about how it is going to be deployed. For instance, if it is going to be deployed in a resource-limited environment, such as a sensor or a robot, then this may place limitations on the complexity of the model. If there are time constraints, e.g., a classification of a signal is required within milliseconds, then this also needs to be taken into account when selecting a model. If using deep learning, then energy costs and carbon footprint may be a consideration, and if using large language models (LLMs), there may be further operational costs for hosting or accessing foundation models. Another consideration is how the model is going to be tied into the broader software system within which it is deployed; this procedure is often far from simple.[20] However, emerging approaches such as ML Ops aim to address some of the difficulties; see Kreuzberger et al.[21] for a review and Shankar et al.[22] for a discussion of common challenges when operationalizing ML models.

HOW TO RELIABLY BUILD MODELS

Building models is one of the more enjoyable parts of ML. With modern ML frameworks, it is easy to throw all manner of approaches at your data and see what sticks. However, this can lead to a disorganized mess of experiments that is hard to justify and write up. So, it is important to approach model building in an organized manner, making sure you use data correctly and putting adequate consideration into the choice of models.

Do not allow test data to leak into the training process

It is essential to have data that you can use to measure how well your model generalizes. A common problem is allowing information about these data to leak into the configuration, training, or selection of models (see Figure 1). When this happens, the data no longer provide a reliable measure of generality, and this is a common reason why published ML models often fail to generalize to real-world data. There are a number of ways that information can leak from a test set. Some of these seem quite innocuous. For instance, during data preparation, when information about the means and ranges of variables within the whole dataset is used to carry out variable scaling or imputation—in

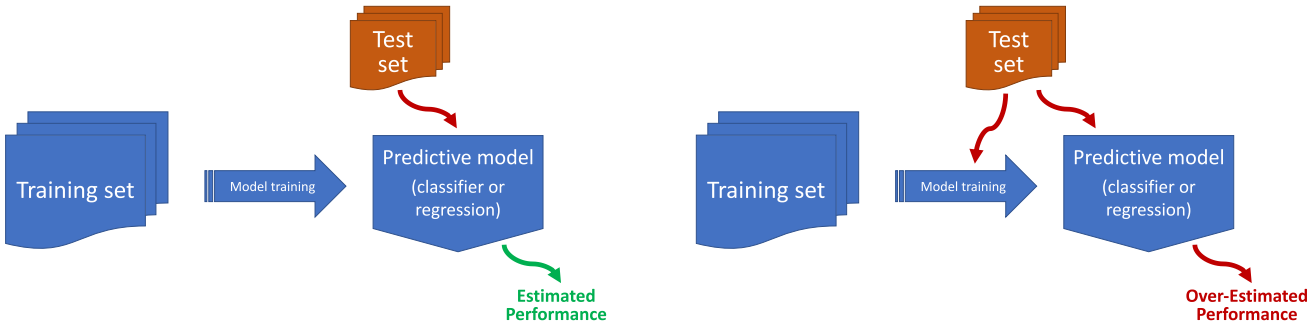


Figure 1. See "do not allow test data to leak into the training process"
(Left) How things should be, with the training set used to train the model and the test set used to measure its generality. (Right) When there is a data leak, the test set can implicitly become part of the training process, meaning that it no longer provides a reliable measure of generality.

order to prevent information leakage, these statistics should be calculated using only the training data. Other common examples of information leakage are carrying out feature selection before partitioning the data (see "do be careful where and how you do feature selection"), using the same test data to evaluate the generality of multiple models (see "do avoid sequential overfitting" and "do not always believe results from community benchmarks"), and applying data augmentation before splitting off the test data (see "do not do data augmentation before splitting your data"). The best thing you can do to prevent these issues is to partition off a subset of your data right at the start of your project and only use this independent test set once to measure the generality of a single model at the end of the project (see "do save some data to evaluate your final model instance"). There are also forms of data leakage that are specific to certain types of data. Time-series data are particularly problematic since the order of samples is significant, and random splits can easily cause leakage and overfitting—see "do not ignore temporal dependencies in time-series data" for more on this. Even for non-time-series data, the experimental conditions used to generate datasets may lead to temporal dependencies or other problematic conditions such as duplicated or similar samples—see "do use an appropriate test set" for an example. In order to prevent leakage, these kinds of issues need to be identified and taken into account when splitting data. For a broader discussion of data leakage, see Kapoor and Narayanan.[23]

Do try out a range of different models

Generally speaking, there is no such thing as a single best ML model. In fact, there is proof of this in the form of the "no free lunch" theorem, which shows that no ML approach is any better than any other when considered over every possible problem.[24] So, your job is to find the ML model that works well for your particular problem. There is some guidance on this. For example, you can consider the inductive biases of ML models, that is, the kind of relationships they are capable of modeling. For instance, linear models, such as linear regression and logistic regression, are a good choice if you know there are no important non-linear relationships between the features in your data but a bad choice otherwise. Good-quality research on closely related problems may also be able to point you toward models that work particularly well. However, a lot of the time, you are still left with quite a few choices, and the only way to work out which model is best is to try them all. Fortunately, modern ML libraries, such as scikit-learn[25] in Python, tidymodels[26] in R, and MLJ[27] in Julia, allow you to try out multiple models with only small changes to your code, so there is no reason not to try them all out and find out for yourself which one works best. However, do not use inappropriate models and use a validation set, rather than the test set, to evaluate them (see "do avoid sequential overfitting"). When comparing models, do optimize your model's hyperparameters, evaluate a model multiple times to make sure you are giving them all a fair chance, and correct for multiple comparisons when you publish your results.

Do not use inappropriate models

By lowering the barrier to implementation, modern ML libraries also make it easy to apply inappropriate models to your data. This, in turn, could look bad when you try to publish your results. A simple example of this is applying models that expect categorical features to a dataset containing numerical features, or vice versa. Some ML libraries allow you to do this, but it may result in a poor model due to loss of information. If you really want to use such a model, then you should transform the features first; there are various ways of doing this, ranging from simple one-hot encodings to complex learned embeddings. Other examples of inappropriate model choice include using a classification model where a regression model would make more sense (or vice versa), attempting to apply a model that assumes no dependencies between variables to time-series data, or using a model that is unnecessarily complex (see "do not assume deep learning will be the best approach"). Also, if you are planning to use your model in practice, do think about how your model will be deployed, and do not use models that are not appropriate for your use case.

Do keep up with progress in deep learning (and its pitfalls)

While deep learning may not always be the best solution (see "do not assume deep learning will be the best approach"), if you are going to use deep learning, then it is advisable to try to keep up with recent developments in this fast-moving field. Figure 2 summarizes some of the important developments over time. Multilayer perceptrons and recurrent neural networks (particularly long short-term memory [LSTM]) have been around for some time but have largely been subsumed by newer models

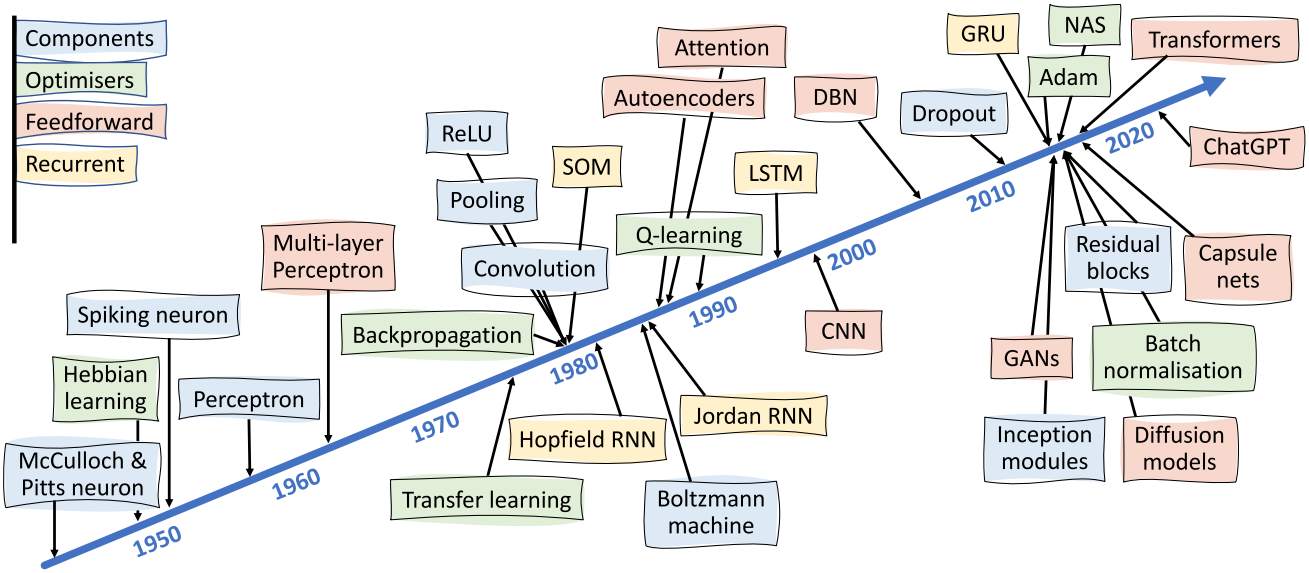


Figure 2. See "do keep up with progress in deep learning (and its pitfalls)"
A rough history of neural networks and deep learning showing what I consider to be the milestones in their development. For a far more thorough account of the field's historical development, take a look at Schmidhuber.[37,38]

such as convolutional neural networks (CNNs)[28,29] and transformers.[30] For example, transformers have become the go-to model for processing sequential data (e.g., natural language) and are increasingly being applied to other data types too, such as images.[31,32] A prominent downside of both transformers and deep CNNs is that they have many parameters and therefore require a lot of data to train them. However, an option for small datasets is to use transfer learning, where a model is pretrained on a large generic dataset and then fine-tuned on the dataset of interest.[33] Larger pretrained models, many of which are freely shared on websites such as Hugging Face, are known as foundation models; see Zhou et al.[34] for a survey. While powerful, these come with their own set of pitfalls. For example, their ability to fully memorize input data is the cause of data security and privacy concerns.[35] The use of opaque, often poorly documented, training datasets also leads to pitfalls when fitting them into broader ML pipelines (see "do combine models (carefully)" for more info) and when comparing them fairly with other ML models (see "do not assume a bigger number means a better model" and "do not always believe results from community benchmarks"). For an extensive yet accessible guide to deep learning, see Zhang et al.[36]

Do not assume deep learning will be the best approach

A common pitfall is to assume that deep neural networks will provide the best solution to any problem and consequently fail to try out other, possibly more appropriate, models. While deep learning is great for certain tasks, it is not good at everything; there are plenty of examples of it being outperformed by "old fashioned" ML models, such as random forests and support vector machines (SVMs). See, for instance, Grinsztajn et al.,[39] who show that tree-based models often outperform deep learners on tabular data. Certain kinds of deep neural network architecture may also be ill-suited to certain kinds of data: see, for example, Zeng et al.,[40] who argue that transformers are not well-suited to time-series forecasting. There are also theoretical reasons why any one kind of model will not always be the best choice (see "do try out a range of different models"). In particular, a deep neural network is unlikely to be a good choice if you have limited data, domain knowledge suggests that the underlying pattern is quite simple, or the model needs to be interpretable. This last point is particularly worth considering: a deep neural network is essentially a very complex piece of decision-making that emerges from interactions between a large number of non-linear functions. Non-linear functions are hard to follow at the best of times, but when you start joining them together, their behavior gets very complicated very fast. While explainable AI (XAI) methods (see "do look at your models") can shine some light on the workings of deep neural networks, they can also mislead you by ironing out the true complexities of the decision space.[41] For this reason, you should take care when using either deep learning or XAI for models that are going to make high-stakes or safety-critical decisions; see Rudin[19] for more on this.

Do be careful where and how you do feature selection

A common stage of training a model is to carry out feature selection (surveyed by Cai et al.[42]). When doing this, it is important to treat it as part of model training and not something more general that you do before model training. A particularly common error is to do feature selection on the whole dataset before splitting off the test set, something that will result in information leaking from the test set into the training process (see "do not allow test data to leak into the training process"). Instead, you should only use the training set to select the features that are used in both the training set and the test set (see Figure 3). The same is true when doing dimensionality reduction. For example, if you are using principal-component analysis, then the component weightings should be determined by looking only at the training data; the same weightings should then be applied to

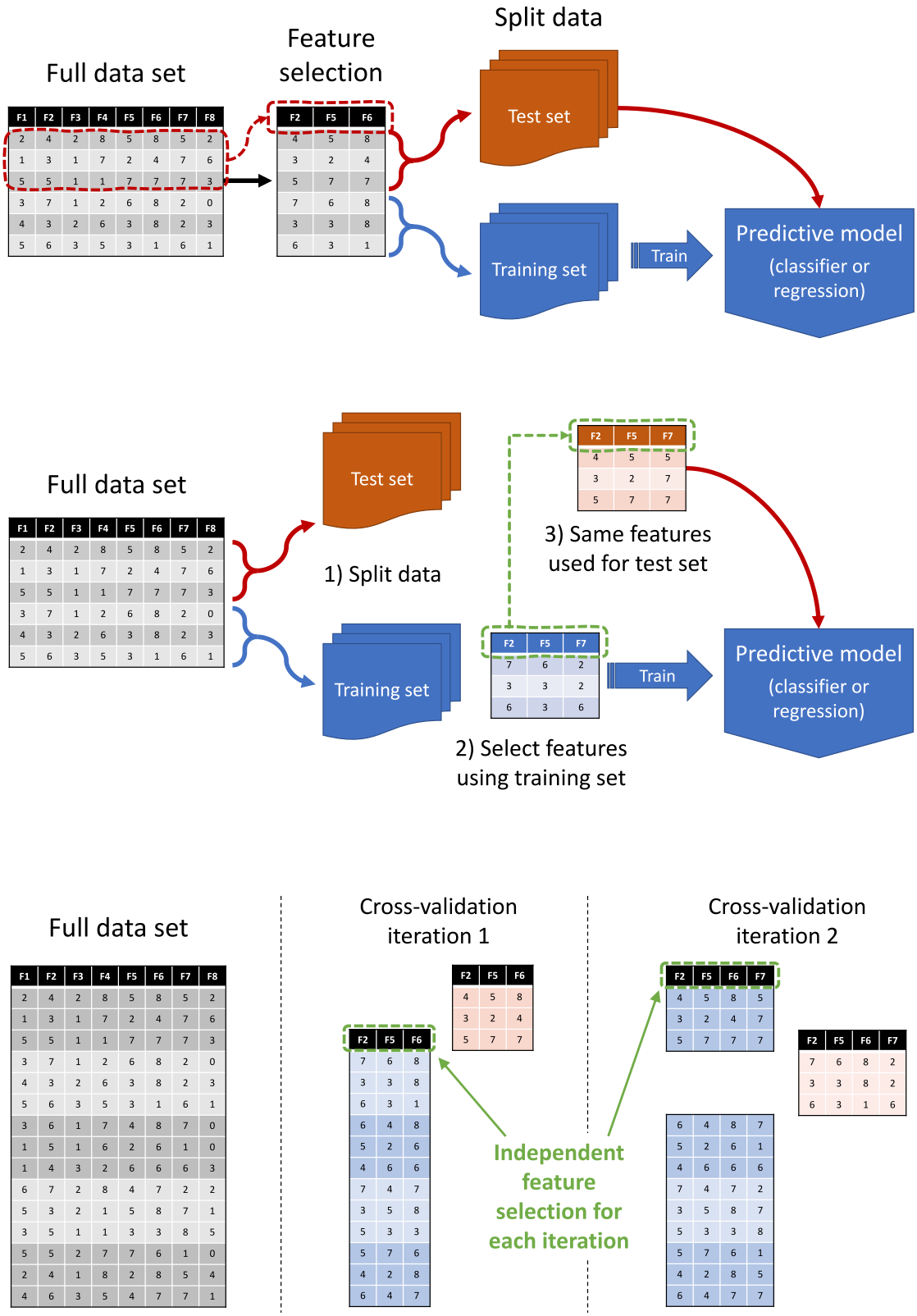


Figure 3. See "do be careful where and how you do feature selection"

(Top) Data leakage due to carrying out feature selection before splitting off the test data (outlined in red), causing the test set to become an implicit part of model training. (Middle) How it should be done. (Bottom) When using cross-validation, it is important to carry out feature selection independently for each iteration, based only on the subset of data (shown in blue) used for training during that iteration.

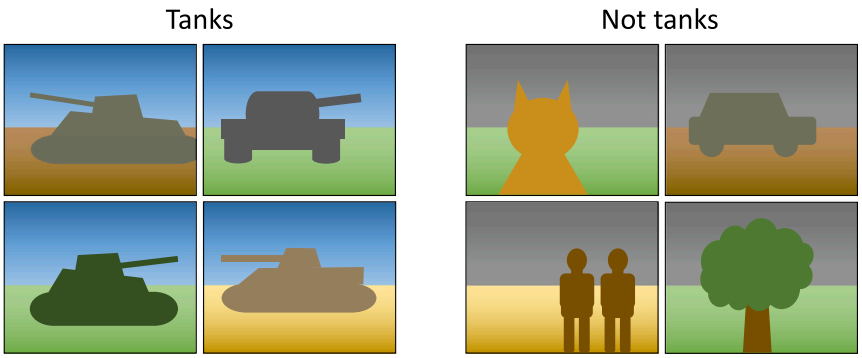


Figure 4. See "do avoid learning spurious correlations"
The problem of spurious correlations in images as illustrated by the tank problem. The images on the left are tanks, and those on the right are not tanks. However, the consistent background (blue for tanks, gray for others) means that these images can be classified by merely looking at the colors of pixels toward the top of the images rather than having to recognize the objects in the images, resulting in a poor model.

the test set. Special care should be taken when using autoencoders for dimensionality reduction—see "do combine models (carefully)." If you are doing CV (see "do evaluate a model multiple times") then it is important to carry out feature selection or dimensionality reduction independently within each iteration, each time using just the training folds (see Figure 3, bottom).

Do optimize your model's hyperparameters

Many models have hyperparameters—that is, numbers or settings that affect the configuration of the model. Examples include the kernel function used in an SVM, the number of trees in a random forest, and the architecture of a neural network. Many of these hyperparameters significantly affect the performance of the model, and there is generally no one-size-fits-all approach when selecting hyperparameters. That is, they need to be fitted to your particular dataset in order to get the most out of the model. While it may be tempting to fiddle around with hyperparameters until you find something that works, this is not likely to be an optimal approach. It is much better to use some kind of hyperparameter optimization strategy, and this is much easier to justify when you write it up. Basic strategies include random search and grid search, but these do not scale well to large numbers of hyperparameters or to models that are expensive to train, so it is worth using tools that search for optimal configurations in a more intelligent manner. See Bischl et al.[43] for further guidance. It is also possible to use AutoML techniques to optimize both the choice of model and its hyperparameters, in addition to other parts of the ML pipeline—see Barbudo et al.[44] for a review.

Do avoid learning spurious correlations

Spurious correlations are features within data that are correlated with the target variable but have no semantic meaning. They are basically red herrings, and it is not uncommon for ML models to pick up on them in training and, consequently, fail to generalize well. A classic example is the tank problem. Legend has it that the US military was looking to train an ML model that could recognize tanks (though there is some debate about whether this actually happened[45]). However, because the tank pictures used in training were taken during different weather conditions than the non-tank pictures, the model ended up discriminating based on features such as the number of blue pixels in the sky rather than the presence of a tank (see Figure 4 for an illustration). An ML model that uses such spurious correlations to perform classification would appear to be very good in terms of its metric scores but would not work in practice. More complex data tend to contain more of these spurious correlations, and more complex models have more capacity to overfit spurious correlations. This means that spurious correlations are a particular issue for deep learning, where approaches such as regularization (see "do keep up with progress in deep learning (and its pitfalls)") and data augmentation (see "do make sure you have enough data") can help mitigate against this. However, spurious correlations can occur in all datasets and models, so it is always worth looking at your trained model to see whether it is responding to appropriate features within your data—see "do look at your models."

HOW TO ROBUSTLY EVALUATE MODELS

In order to contribute to progress in your field, you need to have valid results that you can draw reliable conclusions from. Unfortunately, it is really easy to evaluate ML models unfairly and, by doing so, muddy the waters of academic progress. So, think carefully about how you are going to use data in your experiments, measure the true performance of your models, and report this performance in a meaningful and informative way.

Do use an appropriate test set

First of all, always use a test set to measure the generality of an ML model. How well a model performs on the training set is almost meaningless, and a sufficiently complex model can entirely learn a training set yet capture no generalizable knowledge. It is also important to make sure the data in the test set are appropriate. That is, they should not overlap with the training set, and they should be representative of the wider population. For example, consider a photographic dataset of objects where the images in the training and test set were collected outdoors on a sunny day. The presence of the same weather conditions means that the test set will not be independent, and by not capturing a broader variety of weather conditions, it will also not be representative. Similar situations can occur when a single piece of equipment is used to collect both the training and test data; if the model overlearns characteristics of the equipment, it will likely not generalize to other pieces of equipment, and this will not be detectable by evaluating it on the test set. If using public datasets to test a model, be wary of Frankenstein datasets, which are assembled from other public datasets and risk overlap with training data. Also, be careful when handling datasets that contain multiple data points for each subject; if using these, it is important to make sure that each subject's data points

are kept together when splitting off the test set or doing CV. See Roberts et al.[46] for a revealing account of how a number of these pitfalls led to the failure of the vast majority of COVID-19 detection models to generalize beyond their test sets.

Do not do data augmentation before splitting your data

Data augmentation (see "do make sure you have enough data") can be a useful technique for balancing datasets and boosting the generality and robustness of ML models. However, it is important to do data augmentation only on the training set and not on data that are going to be used for testing. Including augmented data in the test set can lead to a number of problems. One problem is that the model may overfit the characteristics of the augmented data rather than the original samples, and you will not be able to detect this if your test set also contains augmented data. A more critical problem occurs when data augmentation is applied to the entire dataset before it is split into training and test sets. In this scenario, augmented versions of training samples may end up in the test set, which, in the worst case, can lead to a particularly nefarious form of data leakage in which the test samples are mostly variants of the training samples. For an interesting study of how this problem affected an entire field of research, see Vandewiele et al.[47,48]

Do avoid sequential overfitting

Oddly, one of the most pernicious forms of data leakage does not have a commonly agreed upon name (though Hosseini et al.[49,50] suggested "over-hyping," from overfitting of hyperparameters), so I am going to refer to it as sequential overfitting. This occurs when you train multiple models in succession using knowledge gained about each model's performance to guide the configuration of the next one, and you use the same test set to evaluate each model. Often, this is done as an informal process, trying out different models and different hyperparameters until you get good performance on the test set. As such, it is rarely documented, which is one reason why it is so pernicious. Specifically, the problem lies in using the test set throughout this process, as using the test set to choose between models means that information about the test set implicitly leaks into the training process. See Figure 5 for an illustration of this idea. The consequence is that models gradually overfit the test set; the more times you use the test set, the more the overfitting that occurs. The solution is to either use a validation set (i.e., a separate set of samples that are not directly used in training but are used to guide training) or use a holdout dataset to test the final model. See Cawley and Talbot[51] and Hosseini et al.[49,50] for more on this.

Do evaluate a model multiple times

Many ML models are stochastic or unstable. That is, if you train them multiple times, or if you make small changes to the training data, then their performance varies significantly. The same is true of using LLMs at inference time. This means that a single evaluation of a model can be unreliable and may either underestimate or overestimate the model's true potential. For this reason, it is common to carry out multiple evaluations. At training time, there are numerous ways of doing this. For stochastic models, the simplest way is to train the same model multiple times using different random seeds and then look at the average performance. A more robust approach is to also vary the data for each model trained. CV is a particularly popular way of doing this and comes in numerous flavors,[52] most of which involve splitting the data into a number of folds. When doing CV, it is important to be aware of any dependencies within the data and take these into account. Failure to do so can result in data leakage. For instance, in medical datasets, it is commonplace to have multiple data points for a single subject; to avoid data leakage, these should be kept together within the same fold. Time-series data are particularly problematic for CV; see "do not ignore temporal dependencies in time-series data" for a discussion of how to handle this. If you are carrying out hyperparameter optimization, then you should use nested CV (also known as double CV), which uses an extra loop inside the main CV loop to avoid overfitting the test folds. If some of your data classes are small, then you may need to do stratification, which ensures that each class is adequately represented in each fold. In addition to looking at the average performance across multiple evaluations, it is also standard practice to provide some measure of spread or confidence, such as the standard deviation or the 95% confidence interval.

Do save some data to evaluate your final model instance

I have used the term "model" quite loosely, but there is an important distinction between evaluating the potential of a general model (e.g., how well a neural network can solve your problem) and the performance of a particular model instance (e.g., a specific neural network produced by one run of backpropagation). CV (see "do evaluate a model multiple times") is good at the former, but it is less useful for the latter. Say, for instance, that you carried out 10-fold CV. This would result in ten model instances. Say you then select the instance with the highest test fold score as the model that you will use in practice. How do you report its performance? Well, you might think that its test fold score is a reliable measure of its performance, but it probably is not. First, the amount of data in a single fold is relatively small. Second, the instance with the highest score could well be the one with the easiest test fold, so the evaluation data it contains may not be representative. Consequently, the only way of getting a reliable estimate of the model instance's generality may be to use another test set. This is also true in situations where the independence of the existing test set may have been compromised, e.g., by using it more than once (see "do avoid sequential overfitting"). So, if you have enough data, it is better to keep some aside and only use them once to provide an unbiased estimate of the final selected model instance. However, it is worth noting one other option when using CV: ensemble the model instances (see "do combine models (carefully)"). The resulting ensemble will have performance in line with the average as measured through CV, so another test set is not required to measure its performance. On the downside, it will likely have poorer inference time, efficiency, and interpretability than a single model instance, so this approach is generally only worth considering if you have very few data.

Do choose metrics carefully

Be careful which metrics you use to evaluate your ML models. For instance, in the case of classification models, the most commonly used metric is accuracy, which is the proportion of samples in the dataset that were correctly classified by the

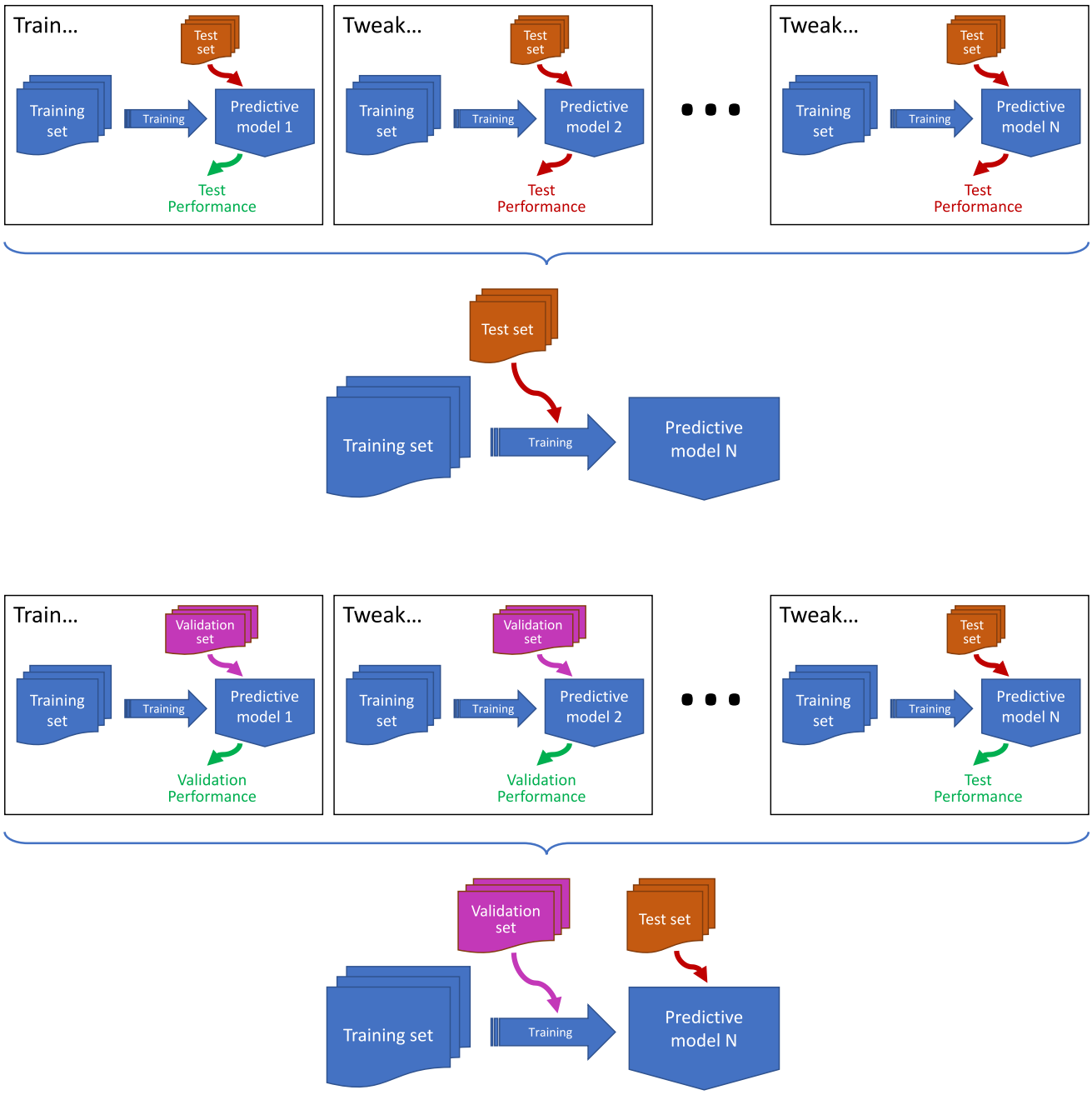


Figure 5. See "do avoid sequential overfitting"

(Top) Using the test set repeatedly during model selection results in the test set becoming an implicit part of the training process. (Bottom) A validation set should be used instead during model selection, and the test set should only be used once to measure the generality of the final model.

model. This works fine if your classes are balanced, i.e., if each class is represented by a similar number of samples within the dataset. But many datasets are not balanced, and in this case, accuracy can be a very misleading metric. Consider, for example, a dataset in which 90% of the samples represent one class and 10% of the samples represent another class. A binary classifier that always outputs the first class, regardless of its input, would have an accuracy of 90% despite being completely useless (see Figure 6). In this kind of situation, it would be preferable to use a metric such as F_1 score, Cohen's kappa coefficient, or Matthew's correlation coefficient, all of which are relatively insensitive to class size imbalance. For a broader review of methods for dealing with imbalanced data, see Haixiang et al.[18] There are also various pitfalls associated with regression metrics, particularly within the context of time-series forecasting; see Hewamalage et al.[53] for a discussion of these. A well-known example is relying only on the root-mean-square error, which (a bit like accuracy) is susceptible to assigning high value to



Figure 6. See "do choose metrics carefully"
The problem with using accuracy as a performance metric on imbalanced data. Here, a dummy model that always predicts the same class label has an accuracy of 50% or 90% depending on the distribution of class labels within the data.

models that always predict no change. See also "do report performance in multiple ways."

Do consider model fairness

Overall performance metrics are not the only important measures of how good a model is. If a model is to be deployed within the real world, then another important measure is fairness. There are various definitions of fairness, but in a nutshell, it is about making sure that the model does not treat its human subjects unequally with regard to characteristics such as gender, ethnicity, income, or personal politics. This is also referred to as algorithmic bias, and there are many examples of models being biased toward or against particular groups of people. A common source of unfairness is using an unrepresentative dataset to train an ML model. For instance, if a medical diagnosis model is trained on data from a single country, then the data may be biased toward the majority ethnicity, and the model may not operate fairly when exposed to users from other ethnicities. However, unfairness can also come from other sources, including subconscious bias during data preparation and the inductive biases of the model. Regardless of the source, it is important to understand any resulting biases and, ideally, take steps to mitigate against them (e.g., applying data augmentation to minority samples—see "do make sure you have enough data"). There are many different fairness metrics, so part of the puzzle is working out which are most relevant to your modeling context; see Caton and Haas[54] for a review.

Do not ignore temporal dependencies in time-series data

Time-series data are unlike many other kinds of data in that the order of the data points is important. Many of the pitfalls in handling time-series data are a result of ignoring this fact. Most notably, time-series data are subject to a particular kind of data leakage (see "do not allow test data to leak into the training process") known as look-ahead bias. This occurs when some or all of the data points used to train the model occur later in the time series than those used to test the model. In effect, this can allow knowledge of the future to leak into training, and this can then bias the test performance. A situation where this commonly occurs is when standard CV (see "do evaluate a model multiple times") is applied to time-series data, as it results in the training folds in all but one of the CV iterations containing data that are in the future relative to the test fold. This can be avoided by using special forms of CV that respect temporal dependencies, such as blocked CV, though whether this is necessary depends, to some extent, on the nature of the time-series data, e.g., whether it is stationary or non-stationary. See Cerqueira et al.[55] and Wang and Ruf[56] for more on this. Look-ahead bias can also result from carrying out data-dependent preprocessing operations before splitting off the test data; see Figure 7 for a simple example of this, but also see "do be careful where and how you do feature selection."

HOW TO COMPARE MODELS FAIRLY

Comparing models is the basis of academic research, but it is surprisingly difficult to get it right. If you carry out a comparison unfairly and publish it, then other researchers may subsequently be led astray. So, do make sure that you evaluate different models within the same context, explore multiple perspectives, and make correct use of statistical tests.

Do not assume a bigger number means a better model

It is not uncommon for a paper to state something like "in previous research, accuracies of up to 94% were reported. Our model achieved 95% and is therefore better." There are various reasons why a higher figure does not imply a better model. For instance, if the models were trained or evaluated on different partitions of the same dataset, then small differences in performance may be due to this. If the datasets had different degrees of class imbalance, then the difference in accuracy could merely reflect this (see "do choose metrics carefully"). If they used different datasets entirely, then this may account for even large differences in performance. Another reason for unfair comparisons is the failure to carry out the same amount of hyperparameter optimization (see "do optimize your model's hyperparameters") when comparing models; for instance, if one model has default settings and the other has been optimized, then the comparison will not be fair. For these reasons and others, comparisons based on published figures should always be treated with caution. To be sure of a fair comparison between two

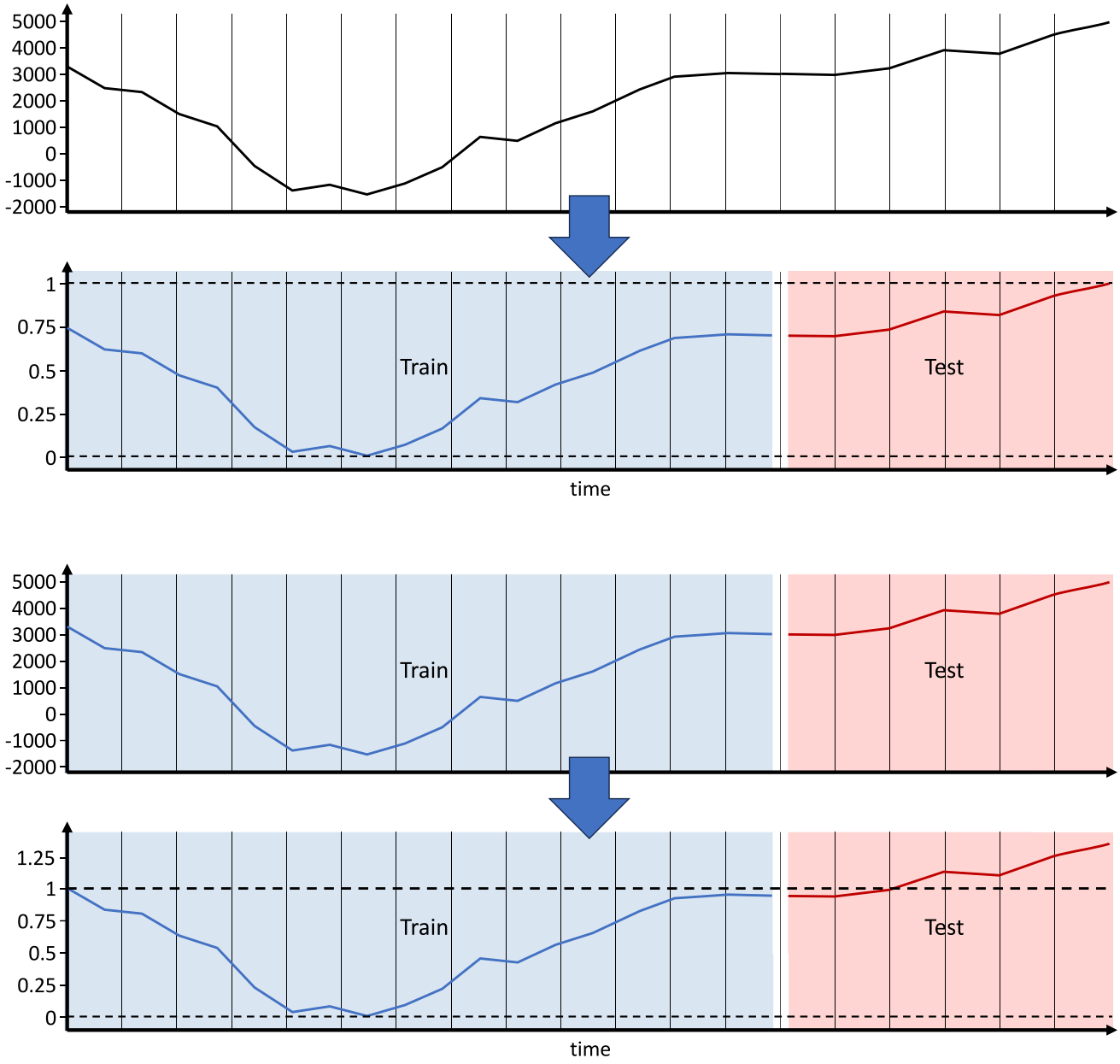


Figure 7. See "do not ignore temporal dependencies in time-series data"

(Top) A time series is scaled to the interval [0, 1] before splitting off the test data (shown in red). This could allow the model to infer that values will increase in the future, causing a potential look ahead bias. (Bottom) Instead, the data should be split before doing scaling so that information about the range of the test data cannot leak into the training data.

approaches, you should freshly implement all the models you are comparing, optimize each one to the same degree, carry out multiple evaluations (see "do evaluate a model multiple times"), and then use statistical tests (see "do use statistical tests when comparing models") to determine whether the differences in performance are significant. A further complication when comparing foundation models (see "do keep up with progress in deep learning (and its pitfalls)") it that the original training data are often unknown; consequently, it may be impossible to ensure that the test set is independent of the training data and, therefore, a fair basis for comparison.

Do use meaningful baselines

When introducing a new modeling approach, it is essential to compare against established approaches. These are commonly referred to as baseline models or just baselines. It is important that these baselines are selected so that they provide a meaningful basis for comparison. Baselines are often simpler than the new approach and are chosen to demonstrate that any complexity in the new model is necessary. For example, if you are extending model X, then it makes sense to use model X as a baseline. However, it also makes sense to use other, simpler models. For instance, if you are developing a deep learning

approach that uses tabular data, then you should also compare against simpler models like decision trees and SVMs to show that a more complex approach is justified. If you are solving a regression problem, then you should also consider using simple baselines like logistic regression. The simplest baselines are known as naive baselines and used to show that your model is not doing something trivial. An illustrative example of why these are necessary is described in Hewamalage et al.,[53] where a complex transformer model designed for time-series forecasting is shown to perform worse than a naive baseline that always forecasts the next value in a time series to be the same as the previous value. This kind of naive baseline, in which there is no real decision-making process, is also known as a dummy model. Another example is a classifier that always outputs the most frequent class label (as described in "do choose metrics carefully"). In addition to simple baselines, it is also important to compare against state-of-the-art models. Otherwise, you may be asked something like "why are you extending model X when model Y is known to be better than model X?"

Do use statistical tests when comparing models

If you want to convince people that your model is better than someone else's, then a statistical test can be a useful tool. Broadly speaking, there are two categories of tests for comparing individual ML models. The first is used to compare individual model instances, e.g., two trained decision trees. For example, McNemar's test is a fairly common choice for comparing two classifiers and works by comparing the classifiers' output labels for each sample in the test set (so do remember to record these). The second category is tests that are used to compare two models more generally, e.g., whether a decision tree or a neural network is a better fit for the data. These require multiple evaluations of each model (see "do evaluate a model multiple times"), which you can get by using CV or repeated resampling (or, if your training algorithm is stochastic, multiple repeats using the same data). The test then compares the two resulting distributions. The Student's t test is a common choice for this kind of comparison, but it is only reliable when the distributions are normally distributed, which is often not the case. A safer bet is Mann-Whitney's U test, as this does not assume that the distributions are normal. For more information, see Raschka[57] and Carrasco et al.[58] See also "do correct for multiple comparisons" and "do be careful when reporting statistical significance."

Do correct for multiple comparisons

Things get a bit more complicated when you want to use statistical tests to compare more than two models, as doing multiple pairwise tests is a bit like using the test set multiple times—it can lead to overly optimistic interpretations of significance. Basically, each time you carry out a comparison between two models using a statistical test, there is a probability that it will discover significant differences where there are not any. This is represented by the confidence level of the test, usually set at 95%: meaning that 1 in 20 times, it will give you a false positive. For a single comparison, this may be a level of uncertainty that you can live with. However, it accumulates. That is, if you do 20 pairwise tests with a confidence level of 95%, then one of them is likely to give you the wrong answer. This is known as the multiplicity effect and is an example of a broader issue in data science known (at least when done intentionally) as data dredging or p-hacking.[59] To address this problem, you can apply a correction for multiple tests. The most common approach is the Bonferroni correction, a very simple method that lowers the significance threshold based on the number of tests that are being carried out; see Salzberg[60] for a gentle introduction. However, there are numerous other approaches, and there is also some debate about when and where these corrections should be applied; for an accessible overview, see Streiner.[61]

Do not always believe results from community benchmarks

In certain problem domains, it has become commonplace to use benchmark datasets to evaluate new ML models. The idea is that because everyone is using the same data to train and test their models, then comparisons will be more transparent. Unfortunately, this approach has some major drawbacks. First, if access to the test set is unrestricted, then you cannot assume that people have not used it as part of the training process. This is known as "training to the test set" and leads to results that are heavily overoptimistic. A more subtle problem is that even if everyone who uses the data only uses the test set once, collectively, the test set is being used many times by the community. In effect, by comparing lots of models on the same test set, it becomes increasingly likely that the best model just happens to overfit the test set and does not necessarily generalize any better than the other models (see "do correct for multiple comparisons" and "do avoid sequential overfitting"). For these and other reasons, you should be careful how much you read into results from a benchmark dataset, and do not assume that a small increase in performance is significant. This is particularly the case where foundation models (see "do keep up with progress in deep learning (and its pitfalls)") are used, as it is possible that their training data included the test sets from community benchmarks. See Paullada et al.[11] for a wider discussion of issues surrounding the use of shared datasets. See also "do report performance in multiple ways."

Do combine models (carefully)

While this section focuses on comparing models, it is good to be aware that ML is not always about choosing between models. Often, it makes sense to use combinations of models. Different ML models explore different trade-offs; by combining them, you can sometimes compensate for the weaknesses of one model by using the strengths of another model, and vice versa. Ensembles are a well-established group of composite models. There are lots of ensemble learning approaches—see Dong et al.[62] for a review—but they can be roughly divided into those that form ensembles out of the same base model type (examples include random forests, bagging, and boosting) and those that combine different types of ML models. An example of the latter is stacked generalization (or stacking), where a model is trained to aggregate the outputs of a group of base models. However, ensembles are not the only kind of composition. Another, increasingly common form of composition occurs when embedding models (such as autoencoders or foundation models such as BERT) are used to provide input to other models. When using stacking or embedding, it is important to ensure that no data

leaks (see "do not allow test data to leak into the training process") occur, i.e., that the test data used to measure the performance of the composite model are not used in the training of any of its components. This is a common pitfall, especially when the model components are trained on overlapping data. To reduce the likelihood of sequential overfitting (see "do avoid sequential overfitting"), it is also advisable to use a separate test set to evaluate the composite model.

HOW TO REPORT YOUR RESULTS

The aim of academic research is not self-aggrandizement but rather having an opportunity to contribute to knowledge. In order to effectively contribute to knowledge, you need to provide a complete picture of your work, covering both what worked and what did not. ML is often about trade-offs—it is very rare that one model is better than another in every way that matters—and you should try to reflect this with a nuanced and considered approach to reporting results and conclusions.

Do be transparent

First of all, always try to be transparent about what you have done and what you have discovered, as this will make it easier for other people to build upon your work. In particular, it is good practice to share your models in an accessible way. For instance, if you used a script to implement all your experiments, then share the script when you publish the results. This means that other people can easily repeat your experiments, which adds confidence to your work. It also makes it a lot easier for people to compare models since they no longer have to reimplement everything from scratch in order to ensure a fair comparison. Knowing that you will be sharing your work also encourages you to be more careful, document your experiments well, and write clean code, which benefits you as much as anyone else. It is also worth noting that issues surrounding reproducibility are gaining prominence in the ML community, so in the future, you may not be able to publish work unless your workflow is adequately documented and shared—for example, see Pineau et al.[63] Checklists ("do use an ML checklist") are useful for knowing what to include in your workflow. You might also find experiment tracking frameworks, such as MLflow,[64] useful for recording your workflow.

Do report performance in multiple ways

One way to achieve better rigor when evaluating and comparing models is to use multiple datasets. This helps to overcome any deficiencies associated with individual datasets (see "do not always believe results from community benchmarks") and allows you to present a more complete picture of your model's performance. It is also good practice to report multiple metrics for each dataset since different metrics can present different perspectives on the results and increase the transparency of your work. For example, if you use accuracy, it is also a good idea to include metrics that are less sensitive to class imbalances (see "do choose metrics carefully"). In domains such as medicine and security, it is important to know where errors are being made; for example, when your model gets things wrong, is it more inclined to false positives or false negatives? Metrics that summarize everything in one number, such as accuracy, give no insight into this. So, it is important to also include partial metrics such as precision and recall or sensitivity and specificity, as these do provide insight into the types of errors your model produces. And make sure it is clear which metrics you are using. For instance, if you report F-scores, be clear whether this is F_1 or some other balance between precision and recall. If you report AUC, indicate whether this is the area under the ROC curve or the PR curve. For a broader discussion, see Blagec et al.[65]

Do not generalize beyond the data

It is important not to present invalid conclusions, as this can lead other researchers astray. A common mistake is to make general statements that are not supported by the data used to train and evaluate models. For instance, if your model does really well on one dataset, then this does not mean that it will do well on other datasets. While you can get more robust insights by using multiple datasets (see "do report performance in multiple ways"), there will always be a limit to what you can infer from any experimental study. There are numerous reasons for this,[11] many of which are to do with how datasets are curated. One common issue is bias, or sampling error: that the data are not sufficiently representative of the real world. Another is overlap: multiple datasets may not be independent and may have similar biases. There is also the issue of quality, and this is a particular issue in deep learning datasets, where the need for quantity of data limits the amount of quality checking that can be done. So, in short, do not overplay your findings, and be aware of their limitations.

Do be careful when reporting statistical significance

I have already discussed statistical tests (see "do use statistical tests when comparing models") and how they can be used to determine differences between ML models. However, statistical tests are not perfect. Some are conservative and tend to underestimate significance; others are liberal and tend to overestimate significance. This means that a positive test does not always indicate that something is significant, and a negative test does not necessarily mean that something is not significant. Then, there is the issue of using a threshold to determine significance; for instance, a 95% confidence threshold (i.e., when $p < 0.05$) means that 1 in 20 times a difference is flagged as significant, it will not be significant. In fact, statisticians are increasingly arguing that it is better not to use thresholds and instead just report p values and leave it to the reader to interpret these.[66] Beyond statistical significance, another thing to consider is whether the difference between two models is actually important. If you have enough samples, you can always find significant differences, even when the actual difference in performance is miniscule. To give a better indication of whether something is important, you can measure effect size. There are a range of approaches used for this: Cohen's d statistic is probably the most common, but more robust approaches, such as Kolmogorov-Smirnov, are preferable. For more on effect size and reporting statistical significance, see Aguinis et al.[67] You might also consider using Bayesian statistics; although there is less guidance and tool support available, these theoretically have a lot going for them, and they avoid many of the pitfalls associated with traditional statistical tests—see Benavoli et al.[68] for more info.

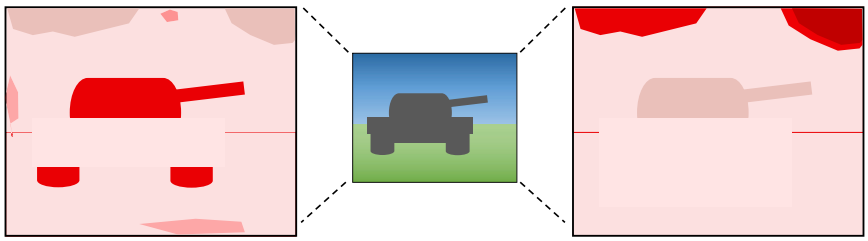


Figure 8. See "do look at your models"
Using saliency maps to analyze vision-based deep learning models. Imagine these two maps (in red) were generated for the image shown in the center, for two different deep learning models trained on the kind of tank recognition data mentioned in "do avoid learning spurious correlations." Darker colors indicate features that are of greater importance to the model, so the model on the left (which predominantly focuses on the components of the tank) is likely to generalize much better than the one on the right (which predominantly focuses on the background of the image).

Do look at your models

Trained models contain a lot of useful information. Unfortunately, many authors just report the performance metrics of a trained model without giving any insight into what it actually learnt. Remember that the aim of research is not to get a slightly higher accuracy than everyone else. Rather, it is to generate knowledge and understanding and share this with the research community. If you can do this, then you are much more likely to get a decent publication out of your work. So, do look inside your models and try to understand how they reach a decision. For relatively simple models like decision trees, it can also be beneficial to provide visualizations of your models, and most libraries have functions that will do this for you. For more complex models, there are a range of XAI techniques that can be used. Some of these are model specific, and others are model agnostic. Well-established examples of the latter are local-interpretable model-agnostic explanations (LIME) and Shapley additive explanations (SHAP); both give insights into which features are important for a model. For CNNs and vision transformers, a common approach is to use saliency maps, which show the importance of different parts of an input image—see Figure 8 for an illustrative example. Grad-CAM is a popular technique for generating these, but there are plenty of other methods too. For non-vision transformers, a common approach is to visualize attention weights. See Dwivedi et al.[69] for a survey of XAI techniques and Ali et al.[70] for a discussion of the limitations of current approaches. While XAI techniques can give you useful insights into a model's behavior, it is important to bear in mind that they are unlikely to tell you exactly what a model is doing. This is particularly the case for deep learning models (see "do not assume deep learning will be the best approach"), whose complexity makes their behavior inherently difficult to analyze. For complex models, ablation studies[71] can also be useful. This involves successively removing parts of the model to see what is important and can result in a simpler model that is more amenable to analysis.

Do use an ML checklist

This guide aims to give an appreciation of the main things that can go wrong during ML plus provide some guidance on how to avoid these things from going wrong. Checklists, on the other hand, are designed to take you more formally through the ML pipeline and encourage you to document (and, more importantly, think about) how your implementation decisions support a meaningful outcome. In some domains, e.g., certain fields of medicine, it is compulsory to complete a checklist before submitting a paper for publication. However, beyond their quality assurance role, checklists are arguably most useful at the start of a study when it comes to planning an ML pipeline. Since I am one of the authors, I would particularly encourage you to look at REFORMS,[72] which is a combined checklist and set of consensus-based recommendations for doing ML-based science (although much of it is also applicable to ML practice more generally). Other, more domain-specific checklists are also available.

CONCLUSIONS

ML is becoming an important part of people's lives, yet the practice of ML is arguably in its infancy. There are many easy-to-make mistakes that can cause an ML model to appear to perform well when, in reality, it does not. In turn, this has the potential to misinform when these models are published and the potential to cause harm if these models are ever deployed. This guide describes the most common of these mistakes and also touches upon more general issues of good practice in ML, such as fairness, transparency, and the avoidance of bias. It also offers advice on avoiding these pitfalls. However, new threats continue to emerge as new approaches to ML are developed, and it is therefore important for users of ML to remain vigilant. This is the nature of a fast-moving research area—the theory of how to do ML almost always lags behind the practice, practitioners will always disagree about the best ways of doing things, and what we think is correct today may not be correct tomorrow.

DECLARATION OF INTERESTS

The author declares no competing interests.

REFERENCES

1. Liao, T., Taori, R., Raji, I.D., and Schmidt, L. (2021). Are we learning yet? A meta review of evaluation failures across machine learning. In Thirty-Fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2) https://openreview.net/forum?id=mPducS1MsEK.

2. Gibney, E. (2022). Is AI fuelling a reproducibility crisis in science? Nature *608*, 250–251. https://doi.org/10.1038/d41586-022-02035-w.

3. Stevens, L.M., Mortazavi, B.J., Deo, R.C., Curtis, L., and Kao, D.P. (2020). Recommendations for reporting machine learning analyses in clinical research. Circ. Cardiovasc. Qual. Outcomes *13*, e006556. https://doi.org/10.1161/CIRCOUTCOMES.120.006556.

4. Whalen, S., Schreiber, J., Noble, W.S., and Pollard, K.S. (2022). Navigating the pitfalls of applying machine learning in genomics. Nat. Rev. Genet. *23*, 169–181. https://doi.org/10.1038/s41576-021-00434-9.

5. Zhu, J.-J., Yang, M., and Ren, Z.J. (2023). Machine learning in environmental research: common pitfalls and best practices. Environ. Sci. Technol. *57*, 17671–17689. https://doi.org/10.1021/acs.est.3c00026.

6. Karande, P., Gallagher, B., and Han, T.Y.-J. (2022). A strategic approach to machine learning for material science: How to tackle real-world challenges and avoid pitfalls. Chem. Mater. *34*, 7650–7665. https://doi.org/10.1021/acs.chemmater.2c01333.

7. Van Giffen, B., Herhausen, D., and Fahse, T. (2022). Overcoming the pitfalls and perils of algorithms: A classification of machine learning biases and mitigation methods. J. Bus. Res. *144*, 93–106. https://doi.org/10.1016/j.jbusres.2022.01.076.

8. Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., and Rieck, K. (2022). Dos and don'ts of machine learning in computer security. In 31st USENIX Security Symposium (USENIX Security 22), pp. 3971–3988. https://www.usenix.org/system/files/sec22-arp.pdf.

9. Malik, M.M. (2020). A hierarchy of limitations in machine learning. Preprint at arXiv. https://doi.org/10.48550/arXiv.2002.05193.

10. Lones, M.A. (2021). How to avoid machine learning pitfalls: a guide for academic researchers. Preprint at arXiv. https://doi.org/10.48550/arXiv.2108.02497.

11. Paullada, A., Raji, I.D., Bender, E.M., Denton, E., and Hanna, A. (2021). Data and its (dis)contents: A survey of dataset development and use in machine learning research. Patterns *2*, 100336. https://doi.org/10.1016/j.patter.2021.100336.

12. Cox, V. (2017). Exploratory data analysis. In Translating Statistics to Make Decisions (Springer), pp. 47–74.

13. Emmanuel, T., Maupong, T., Mpoeleng, D., Semong, T., Mphago, B., and Tabona, O. (2021). A survey on missing data in machine learning. J. Big Data *8*, 140. https://doi.org/10.1186/s40537-021-00516-9.

14. Côté, P.-O., Nikanjam, A., Ahmed, N., Humeniuk, D., and Khomh, F. (2024). Data cleaning and machine learning: a systematic literature review. Autom. Software Eng. *31*, 54. https://doi.org/10.1007/s10515-024-00453-w.

15. Côté, P.-O., Nikanjam, A., Ahmed, N., Humeniuk, D., and Khomh, F. (2024). Data cleaning and machine learning: a systematic literature review. Preprint at arXiv. https://doi.org/10.48550/arXiv.2310.01765.

16. Wang, Z., Wang, P., Liu, K., Wang, P., Fu, Y., Lu, C.-T., Aggarwal, C.C., Pei, J., and Zhou, Y. (2024). A comprehensive survey on data augmentation. Preprint at arXiv. https://doi.org/10.48550/arXiv.2405.09591.

17. Iglesias, G., Talavera, E., González-Prieto, Á., Mozo, A., and Gómez-Canaval, S. (2023). Data augmentation techniques in time series domain: a survey and taxonomy. Neural Comput. Appl. *35*, 10123–10145. https://doi.org/10.1007/s00521-023-08459-3.

18. Haixiang, G., Yijing, L., Shang, J., Mingyun, G., Yuanyue, H., and Bing, G. (2017). Learning from class-imbalanced data: Review of methods and applications. Expert Syst. Appl. *73*, 220–239. https://doi.org/10.1016/j.eswa.2016.12.035.

19. Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. Nat. Mach. Intell. *1*, 206–215. https://doi.org/10.1038/s42256-019-0048-x.

20. Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. Adv. Neural Inf. Process. Syst. *28*, 2503–2511. https://papers.nips.cc/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf.

21. Kreuzberger, D., Kühl, N., and Hirschl, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. IEEE Access *11*, 31866–31879. https://doi.org/10.1109/ACCESS.2023.3262138.

22. Shankar, S., Garcia, R., Hellerstein, J.M., and Parameswaran, A.G. (2022). Operationalizing machine learning: An interview study. Preprint at arXiv. https://doi.org/10.48550/arXiv.2209.09125.

23. Kapoor, S., and Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. Patterns *4*, 100804. https://doi.org/10.1016/j.patter.2023.100804.

24. Wolpert, D.H. (2002). The Supervised Learning No-Free-Lunch Theorems. In Soft Computing and Industry, R. Roy, M. Köppen, S. Ovaska, T. Furuhashi, and F. Hoffmann, eds. (Springer), pp. 25–42. https://doi.org/10.1007/978-1-4471-0123-9_3.

25. Varoquaux, G., Buitinck, L., Louppe, G., Grisel, O., Pedregosa, F., and Mueller, A. (2015). Scikit-learn: Machine learning without learning the machinery. GetMobile: Mobile Comput. Commun. *19*, 29–33. https://doi.org/10.1145/2786984.2786995.

26. Kuhn, M., and Wickham, H. (2020). Tidymodels: a collection of packages for modeling and machine learning using tidyverse principles. https://www.tidymodels.org.

27. Blaom, A., Kiraly, F., Lienart, T., Simillides, Y., Arenas, D., and Vollmer, S. (2020). MLJ: A Julia package for composable machine learning. J. Open Source Softw. *5*, 2704. https://doi.org/10.21105/joss.02704.

28. Li, Z., Liu, F., Yang, W., Peng, S., and Zhou, J. (2022). A survey of convolutional neural networks: analysis, applications, and prospects. IEEE Transact. Neural Networks Learn. Syst. *33*, 6999–7019. https://doi.org/10.1109/TNNLS.2021.3084827.

29. Li, Z., Yang, W., Peng, S., and Liu, F. (2020). A survey of convolutional neural networks: analysis, applications, and prospects. Preprint at arXiv. https://doi.org/10.48550/arXiv.2004.02806.

30. Lin, T., Wang, Y., Liu, X., and Qiu, X. (2022). A survey of transformers. AI Open *3*, 111–132. https://doi.org/10.1016/j.aiopen.2022.10.001.

31. Khan, S., Naseer, M., Hayat, M., Zamir, S.W., Khan, F.S., and Shah, M. (2022). Transformers in vision: A survey. ACM Comput. Surv. *54*, 1–41. https://doi.org/10.1145/3505244.

32. Khan, S., Naseer, M., Hayat, M., Zamir, S.W., Khan, F.S., and Shah, M. (2022). Transformers in vision: A survey. Preprint at arXiv. https://doi.org/10.48550/arXiv.2101.01169.

33. Han, X., Zhang, Z., Ding, N., Gu, Y., Liu, X., Huo, Y., Qiu, J., Yao, Y., Zhang, A., Zhang, L., et al. (2021). Pre-trained models: Past, present and future. AI Open *2*, 225–250. https://doi.org/10.1016/j.aiopen.2021.08.002.

34. Zhou, C., Li, Q., Li, C., Yu, J., Liu, Y., Wang, G., Zhang, K., Ji, C., Yan, Q., He, L., et al. (2023). A comprehensive survey on pretrained foundation models: A history from BERT to ChatGPT. Preprint at arXiv. https://doi.org/10.48550/arXiv.2302.09419.

35. Li, H., Chen, Y., Luo, J., Kang, Y., Zhang, X., Hu, Q., Chan, C., and Song, Y. (2023). Privacy in large language models: Attacks, defenses and future directions. Preprint at arXiv. https://doi.org/10.48550/arXiv.2310.10383.

36. Zhang, A., Lipton, Z.C., Li, M., and Smola, A.J. (2023). Dive into Deep Learning (Cambridge University Press). https://d2l.ai.

37. Schmidhuber, J. (2015). Deep learning in neural networks: An overview. Neural Network. *61*, 85–117. https://doi.org/10.1016/j.neunet.2014.09.003.

38. Schmidhuber, J. (2014). Deep learning in neural networks: An overview. Preprint at arXiv. https://doi.org/10.48550/arXiv.1404.7828.

39. Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? Adv. Neural Inf. Process. Syst. *35*, 507–520. https://openreview.net/pdf?id=Fp7__phQszn.

40. Zeng, A., Chen, M., Zhang, L., and Xu, Q. (2023). Are transformers effective for time series forecasting? Proc. AAAI Conf. Artif. Intell. *37*, 11121–11128. https://ojs.aaai.org/index.php/AAAI/article/view/26317/26089.

41. Molnar, C., König, G., Herbinger, J., Freiesleben, T., Dandl, S., Scholbeck, C.A., Casalicchio, G., Grosse-Wentrup, M., and Bischl, B. (2020). General pitfalls of model-agnostic interpretation methods for machine learning models. In International Workshop on Extending Explainable AI Beyond Deep Models and Classifiers (Springer), pp. 39–68. https://doi.org/10.1007/978-3-031-04083-2_4.

42. Cai, J., Luo, J., Wang, S., and Yang, S. (2018). Feature selection in machine learning: A new perspective. Neurocomputing *300*, 70–79. https://doi.org/10.1016/j.neucom.2017.11.077.

43. Bischl, B., Binder, M., Lang, M., Pielok, T., Richter, J., Coors, S., Thomas, J., Ullmann, T., Becker, M., Boulesteix, A.-L., et al. (2023). Hyperparameter

optimization: Foundations, algorithms, best practices, and open challenges. WIREs Data Min. &. Knowl. *13*, e1484. https://doi.org/10.1002/widm.1484.

44. Barbudo, R., Ventura, S., and Romero, J.R. (2023). Eight years of AutoML: categorisation, review and trends. Knowl. Inf. Syst. *65*, 5097–5149. https://doi.org/10.1007/s10115-023-01935-1.

45. Branwen, G. (2011). The neural net tank urban legend. https://gwern.net/tank.

46. Roberts, M., Driggs, D., Thorpe, M., Gilbey, J., Yeung, M., Ursprung, S., Aviles-Rivero, A.I., Etmann, C., McCague, C., Beer, L., et al. (2021). Common pitfalls and recommendations for using machine learning to detect and prognosticate for COVID-19 using chest radiographs and CT scans. Nat. Mach. Intell. *3*, 199–217. https://doi.org/10.1038/s42256-021-00307-0.

47. Vandewiele, G., Dehaene, I., Kovács, G., Sterckx, L., Janssens, O., Ongenae, F., De Backere, F., De Turck, F., Roelens, K., Decruyenaere, J., et al. (2021). Overly optimistic prediction results on imbalanced data: a case study of flaws and benefits when applying over-sampling. Artif. Intell. Med. *111*, 101987. https://doi.org/10.1016/j.artmed.2020.101987.

48. Vandewiele, G., Dehaene, I., Kovács, G., Sterckx, L., Janssens, O., Ongenae, F., De Backere, F., De Turck, F., Roelens, K., Decruyenaere, J., et al. (2020). Overly optimistic prediction results on imbalanced data: a case study of flaws and benefits when applying over-sampling. Preprint at arXiv. https://doi.org/10.48550/arXiv.2001.06296.

49. Hosseini, M., Powell, M., Collins, J., Callahan-Flintoft, C., Jones, W., Bowman, H., and Wyble, B. (2020). I tried a bunch of things: The dangers of unexpected overfitting in classification of brain data. Neurosci. Biobehav. Rev. *119*, 456–467. https://doi.org/10.1016/j.neubiorev.2020.09.036.

50. Powell, M., Hosseini, M., Collins, J., Callahan-Flintoft, C., Jones, W., Bowman, H., and Wyble, B. (2016). I tried a bunch of things: the dangers of unexpected overfitting in classification. Preprint at bioRxiv. https://doi.org/10.1101/078816.

51. Cawley, G.C., and Talbot, N.L. (2010). On over-fitting in model selection and subsequent selection bias in performance evaluation. J. Mach. Learn. Res. *11*, 2079–2107. https://www.jmlr.org/papers/volume11/cawley10a/cawley10a.pdf.

52. Arlot, S., and Celisse, A. (2010). A survey of cross-validation procedures for model selection. Stat. Surv. *4*, 40–79. https://doi.org/10.1214/09-SS054.

53. Hewamalage, H., Ackermann, K., and Bergmeir, C. (2023). Forecast evaluation for data scientists: common pitfalls and best practices. Data Min. Knowl. Discov. *37*, 788–832. https://doi.org/10.1007/s10618-022-00894-5.

54. Caton, S., and Haas, C. (2024). Fairness in machine learning: A survey. ACM Comput. Surv. *56*, 1–38. https://doi.org/10.1145/3616865.

55. Cerqueira, V., Torgo, L., and Mozetič, I. (2020). Evaluating time series forecasting models: An empirical study on performance estimation methods. Mach. Learn. *109*, 1997–2028. https://doi.org/10.1007/s10994-020-05910-7.

56. Ruf, J., and Wang, W. (2022). Information leakage in backtesting. SSRN. https://doi.org/10.2139/ssrn.3836631.

57. Raschka, S. (2020). Model evaluation, model selection, and algorithm selection in machine learning. Preprint at arXiv. https://doi.org/10.48550/arXiv.1811.12808.

58. Carrasco, J., García, S., Rueda, M., Das, S., and Herrera, F. (2020). Recent trends in the use of statistical tests for comparing swarm and evolutionary computing algorithms: Practical guidelines and a critical review. Swarm Evol. Comput. *54*, 100665. https://doi.org/10.1016/j.swevo.2020.100665.

59. Stefan, A.M., and Schönbrodt, F.D. (2023). Big little lies: A compendium and simulation of p-hacking strategies. R. Soc. Open Sci. *10*, 220346. https://doi.org/10.1098/rsos.220346.

60. Salzberg, S.L. (1997). On comparing classifiers: Pitfalls to avoid and a recommended approach. Data Min. Knowl. Discov. *1*, 317–328. https://doi.org/10.1023/A:1009752403260.

61. Streiner, D.L. (2015). Best (but oft-forgotten) practices: the multiple problems of multiplicity—whether and how to correct for many statistical tests. Am. J. Clin. Nutr. *102*, 721–728. https://doi.org/10.3945/ajcn.115.113548.

62. Dong, X., Yu, Z., Cao, W., Shi, Y., and Ma, Q. (2020). A survey on ensemble learning. Front. Comput. Sci. *14*, 241–258. https://doi.org/10.1007/s11704-019-8208-z.

63. Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d'Alché Buc, F., Fox, E., and Larochelle, H. (2021). Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). J. Mach. Learn. Res. *22*, 1–20. https://www.jmlr.org/papers/volume22/20-303/20-303.pdf.

64. Chen, A., Chow, A., Davidson, A., DCunha, A., Ghodsi, A., Hong, S.A., Konwinski, A., Mewald, C., Murching, S., Nykodym, T., et al. (2020). Developments in MLflow: A system to accelerate the machine learning lifecycle. In Proceedings of the Fourth International Workshop on Data Management for End-to-End Machine Learning, pp. 1–4. https://doi.org/10.1145/3399579.3399867.

65. Blagec, K., Dorffner, G., Moradi, M., and Samwald, M. (2020). A critical analysis of metrics used for measuring progress in artificial intelligence. Preprint at arXiv. https://doi.org/10.48550/arXiv.2008.02577.

66. Betensky, R.A. (2019). The p-value requires context, not a threshold. Am. Statistician *73*, 115–117. https://doi.org/10.1080/00031305.2018.1529624.

67. Aguinis, H., Vassar, M., and Wayant, C. (2021). On reporting and interpreting statistical significance and p values in medical research. BMJ Evid. Based. Med. *26*, 39–42. https://doi.org/10.1136/bmjebm-2019-111264.

68. Benavoli, A., Corani, G., Demšar, J., and Zaffalon, M. (2017). Time for a change: a tutorial for comparing multiple classifiers through Bayesian analysis. J. Mach. Learn. Res. *18*, 2653–2688. https://jmlr.org/papers/v18/16-305.html.

69. Dwivedi, R., Dave, D., Naik, H., Singhal, S., Omer, R., Patel, P., Qian, B., Wen, Z., Shah, T., Morgan, G., and Ranjan, R. (2023). Explainable AI (XAI): Core ideas, techniques, and solutions. ACM Comput. Surv. *55*, 1–33. https://doi.org/10.1145/3561048.

70. Ali, S., Abuhmed, T., El-Sappagh, S., Muhammad, K., Alonso-Moral, J.M., Confalonieri, R., Guidotti, R., Del Ser, J., Díaz-Rodríguez, N., and Herrera, F. (2023). Explainable Artificial Intelligence (XAI): What we know and what is left to attain Trustworthy Artificial Intelligence. Inf. Fusion *99*, 101805. https://doi.org/10.1016/j.inffus.2023.101805.

71. Meyes, R., Lu, M., de Puiseau, C.W., and Meisen, T. (2019). Ablation studies in artificial neural networks. Preprint at arXiv. https://doi.org/10.48550/arXiv.1901.08644.

72. Kapoor, S., Cantrell, E.M., Peng, K., Pham, T.H., Bail, C.A., Gundersen, O.E., Hofman, J.M., Hullman, J., Lones, M.A., Malik, M.M., et al. (2024). REFORMS: Consensus-based recommendations for machine-learning-based science. Sci. Adv. *10*, eadk3452. https://doi.org/10.1126/sciadv.adk3452.

About the author

Michael Lones is a professor of computer science at Heriot-Watt University in Edinburgh, UK. He completed his PhD at the University of York in 2003 and has worked in machine learning and optimization for over 20 years, applying these approaches within diverse areas, including biology, medicine, security, control systems, computer vision, and robotics. This has given him ample experience in the things that can go wrong. He has authored around 100 publications and also writes for a more general audience at https://fetchdecodeexecute.substack.com.